Paradigmas y Lenguajes de Programación

Carrera: Licenciatura en Gestión de Recursos Tecnológicos
Universidad Gastón Dachary

Año 2026

Nombre y apellido: Cristian Collins
Número de matrícula/usuario: ugd600249
Carrera: Licenciatura en Recursos Tecnológicos
Materia: Paradigmas y Lenguajes de Programación
Profesores: Ing. Agustín Encina, Ing. Dante Sicardi

UNIVERSIDAD Gastón Dachary

ACTIVIDAD N° 3

Lenguaje elegido: Go

El lenguaje seleccionado es Go (Golang), un lenguaje moderno de propósito general desarrollado por Google en 2009 (lanzado públicamente en 2012). Fue diseñado por Robert Griesemer, Rob Pike y Ken Thompson con foco en tres pilares: simplicidad sintáctica, alto rendimiento y concurrencia nativa. Es compilado, fuertemente tipado y con recolección de basura automática.

Paradigma: Concurrente

Go pertenece principalmente al paradigma concurrente. A diferencia de otros lenguajes que incorporan concurrencia como una característica adicional, en Go la concurrencia es un concepto de primer nivel del lenguaje, presente desde su diseño original. Go implementa el modelo CSP (Communicating Sequential Processes) propuesto por Tony Hoare, donde los procesos se comunican a través de canales en lugar de compartir memoria directamente.

Fundamentación Teórica

El paradigma concurrente permite que múltiples tareas progresen de forma simultánea o intercalada, optimizando el uso de los recursos del sistema. Es importante distinguir dos conceptos relacionados pero diferentes:
Concurrencia vs. Paralelismo:
• **Concurrencia:** múltiples tareas progresan en el tiempo, pero no necesariamente al mismo instante. El scheduler decide cuándo ejecuta cada una (pueden estar en un solo núcleo).
• **Paralelismo**: múltiples tareas se ejecutan literalmente al mismo tiempo, en distintos núcleos del procesador.
Go implementa concurrencia a través de dos mecanismos principales:
Goroutines:
Son funciones que se ejecutan de forma concurrente con otras goroutines en el mismo espacio de direcciones. Se lanzan con la palabra clave go antes de una llamada a función. Son extremadamente livianas (comienzan con unos pocos KB de stack) y el runtime de Go gestiona su planificación automáticamente sobre los hilos del sistema operativo.
Channels:
Son el mecanismo de comunicación entre goroutines. Siguen el principio de Go: "No te comuniques compartiendo memoria; compartí memoria comunicándote". Los channels permiten enviar y recibir valores de forma segura entre goroutines, evitando las race conditions
que surgen al acceder a variables compartidas.
Problemas clásicos de concurrencia:
• **Race conditions**: ocurren cuando dos goroutines acceden a una variable compartida sin

sincronización. Go incluye un detector de carreras integrado (go run -race).
• **Deadlocks**: cuando goroutines se bloquean mutuamente esperando recursos que nunca se liberan. Go detecta deadlocks simples en tiempo de ejecución.
Este paradigma es especialmente relevante en sistemas modernos como servidores web, aplicaciones distribuidas y procesamiento de datos, donde múltiples tareas deben ejecutarse de forma eficiente sin bloquearse entre sí.

Ejemplo practico en Go

El ejemplo implementa dos goroutines que ejecutan tareas de forma concurrente, utilizando sync.WaitGroup para la sincronización y un channel con buffer para la comunicación entre goroutines. Este enfoque es el recomendado en Go ya que garantiza una sincronización correcta sin importar cuánto tarden las tareas.

```
package main
import (
 "fmt"
 "sync"
 "time"
)
// tarea simula un proceso con varios pasos.
// Recibe el WaitGroup para avisar cuando termina,
// y un channel para enviar sus resultados.
func tarea(nombre string, resultado chan<- string, wg *sync.WaitGroup) {
 defer wg.Done() // se ejecuta al salir: resta 1 al WaitGroup
 for i := 1; i <= 3; i++ {
 time.Sleep(time.Millisecond * 500)
 // envía el resultado al channel en lugar de imprimir directo
 resultado <- fmt.Sprintf("%s: paso %d completado", nombre, i)
 }
}
func main() {
 var wg sync.WaitGroup
 // canal con buffer 6: evita que las goroutines se bloqueen
 // al enviar si nadie lee inmediatamente (buffer = 2 goroutines x 3 pasos)
 resultado := make(chan string, 6)
 // registramos 2 goroutines en el WaitGroup
 wg.Add(2)
 go tarea("Proceso 1", resultado, &wg)
 go tarea("Proceso 2", resultado, &wg)
 // goroutine auxiliar: cierra el canal cuando ambas tareas terminan
 // Esto permite que el range de abajo sepa cuándo dejar de leer
 go func() {
 wg.Wait()
 close(resultado)
 }()
 // recibe e imprime todos los mensajes hasta que el canal se cierre
```

```
 for msg := range resultado {
 fmt.Println(msg)
 }
 fmt.Println("--- Fin del programa ---")
}
```

Salida esperada del programa (el orden puede variar):

```
Proceso 1: paso 1 completado
Proceso 2: paso 1 completado
Proceso 2: paso 2 completado
Proceso 1: paso 2 completado
Proceso 1: paso 3 completado
Proceso 2: paso 3 completado
--- Fin del programa –
```

***Nota**: el orden de los mensajes entre Proceso 1 y Proceso 2 puede variar en cada ejecución, lo cual es justamente una característica del paradigma concurrente: no hay una secuencia fija entre tareas independientes*

Análisis del Ejemplo

Sync.WaitGroup — sincronización correcta:
El WaitGroup es la herramienta estándar de Go para esperar a que un conjunto de goroutines finalice. Funciona como un contador: wg.Add(2) indica que se esperan 2 goroutines; cada una llama a wg.Done() al terminar (mediante defer, lo que garantiza que se ejecuta incluso ante un error); y wg.Wait() bloquea hasta que el contador llegue a cero.
Esto es superior al uso de time.Sleep() para esperar goroutines, ya que Sleep usa un tiempo arbitrario que podría ser insuficiente o excesivo. El WaitGroup espera exactamente el tiempo necesario, sin depender de estimaciones.
Channel con buffer — comunicación sin bloqueos:
El channel resultado tiene un buffer de 6 posiciones (2 goroutines × 3 pasos). Esto permite que las goroutines envíen sus mensajes sin bloquearse aunque nadie los esté leyendo en ese momento. Una vez que ambas goroutines terminan, la goroutine auxiliar cierra el canal con close(resultado), lo que hace que el for range del main sepa cuándo terminar de leer.
Concurrencia observada en la salida:
La salida del programa muestra cómo los mensajes de Proceso 1 y Proceso 2 se intercalan de forma no determinista. Esto ilustra la naturaleza del paradigma concurrente: ambas tareas progresan independientemente, y el orden de sus mensajes depende del scheduler de Go en cada ejecución.

Conclusión

El paradigma concurrente es fundamental en el desarrollo de software moderno, permitiendo construir sistemas más eficientes, escalables y con mejor capacidad de respuesta. Go implementa este paradigma de forma elegante y segura a través de sus goroutines y channels, siguiendo el modelo CSP. La elección de sync.WaitGroup junto con channels demuestra las buenas prácticas recomendadas por el propio lenguaje: sincronizar mediante estructuras diseñadas para ello en lugar de soluciones frágiles, y comunicar goroutines mediante canales en lugar de memoria compartida.

Esto reduce significativamente el riesgo de race conditions y deadlocks, que son los principales problemas del paradigma concurrente.

El uso de channels evita compartir memoria directamente, lo que reduce la necesidad de mecanismos tradicionales como mutex o semáforos. Go demuestra que la concurrencia no tiene por qué ser compleja: con pocas primitivas bien diseñadas es posible construir sistemas concurrentes robustos y fáciles de razonar, lo que lo convierte en una herramienta ideal para servidores web, microservicios y sistemas distribuidos